



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 6
із дисципліни «Технології розробки програмного забезпечення»
Тема: «Патерни проектування.»

Виконав
Студенти групи ІА-31:
Корнійчук М.Р

Перевірив:
Мягкий М.Ю

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

Варіант :

3. Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Посилання на гіт: https://github.com/MkEger/trpz-labs/tree/Lab_6

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Хід роботи

Короткий опис виконання роботи:

У рамках виконання лабораторної роботи було спроектовано та реалізовано архітектуру системи «Текстовий редактор» із застосуванням поведінкового патерну проектування Observer (Спостерігач).

Головною проблемою, яка вирішувалася, була необхідність синхронізації стану основного текстового поля з іншими компонентами системи (рядком статистики та модулем логування) без створення жорстких залежностей (High Coupling) між ними.

Для цього було розроблено механізм підписки на події, де:

1. **Суб'єкт (Subject)** - клас-менеджер тексту, який відстежує зміни контенту.
2. **Спостерігачі (Observers)** - незалежні модулі, які автоматично отримують сповіщення про зміни та виконують відповідну бізнес-логіку (оновлення UI, запис у лог).

Тема проекту: Текстовий редактор

1. Реалізувати не менше 3-х класів відповідно до обраної теми

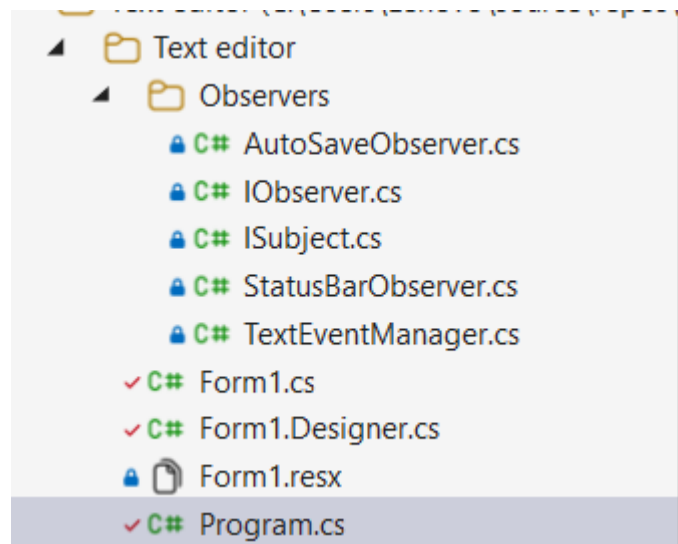


Рис. 1 - Структура проекту

У процесі виконання лабораторної роботи було створено п'ять основних класів, які забезпечують реалізацію реактивної поведінки текстового редактора з використанням шаблону Observer (Рис.1). Наведемо детальний опис кожного класу:

1. **IObserver (інтерфейс)** Базовий інтерфейс для всіх спостерігачів у системі. Визначає контракт для отримання повідомлень про зміни у суб'єкта.
 - `Update(string textContent)` – метод, який викликається суб'єктом (Subject) для передачі оновленого стану (тексту) спостерігачу.
2. **ISubject (інтерфейс)** Інтерфейс суб'єкта, за яким ведеться спостереження. Визначає методи для управління підписками.

- `Attach(IObserver observer)` – додає нового спостерігача до списку підписників.
- `Detach(IObserver observer)` – видаляє спостерігача зі списку.
- `Notify()` – сповіщає всіх підписаних спостерігачів про зміну стану.

3. `TextEventManager (ConcreteSubject)` Клас, що виступає в ролі конкретного суб'єкта. Він керує станом тексту редактора та відповідає за оповіщення підписників. Приватні поля:

- `_observers` – список підписаних спостерігачів.
- `_textContent` – поточний вміст тексту. Основні методи:
- `TextContent` (властивість) – при зміні значення автоматично викликає метод `Notify()`.
- `Notify()` – проходить по списку `_observers` і викликає метод `Update` для кожного з них.

4. `StatusBarObserver (ConcreteObserver)` Конкретна реалізація спостерігача, що відповідає за оновлення інтерфейсу користувача (`StatusStrip`). Функціонал:

- Отримує оновлений текст.
- Підраховує кількість символів та слів.
- Виводить статистику та час останнього оновлення у `ToolStripStatusLabel`.

5. `AutoSaveObserver (ConcreteObserver)` Конкретна реалізація спостерігача, що відповідає за фонові операції (логування/автозбереження). Функціонал:

- Реагує на кожну зміну тексту.
- Імітує процес автозбереження, виводячи повідомлення в консоль налагодження (`Debug`), що дозволяє відстежувати історію змін без блокування основного інтерфейсу.

2. Реалізувати один з розглянутих шаблонів за обраною темою

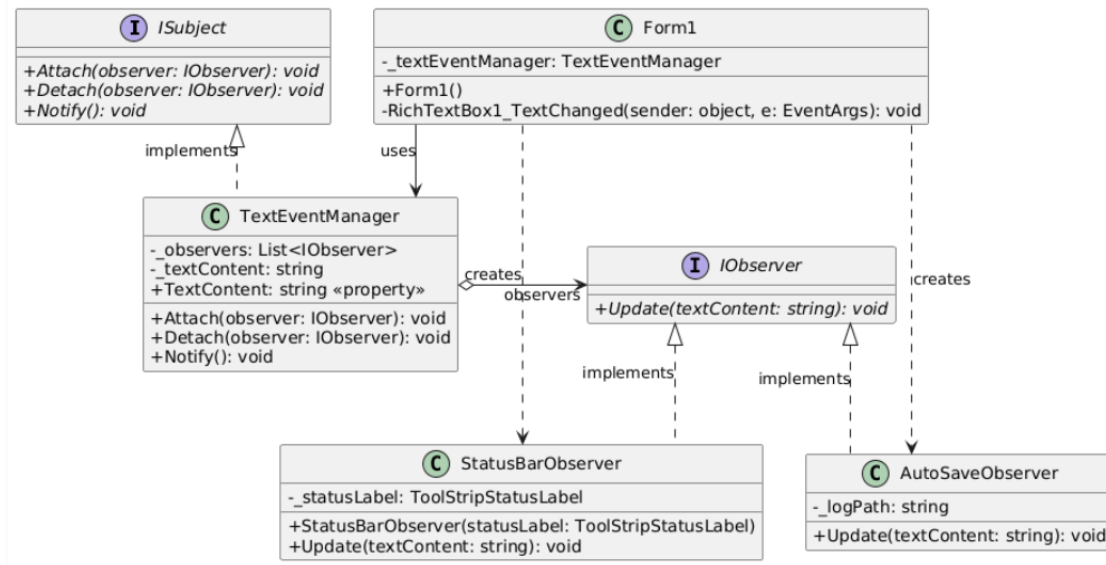


Рис. 2 - Діаграма класів

У рамках проєкту текстового редактора патерн Observer було реалізовано для забезпечення реакції системи на дії користувача в реальному часі без жорсткого зв'язування компонентів.

Реалізація: Основна ідея полягала у відокремленні логіки зберігання даних (**TextEventManager**) від логіки їх відображення та обробки.

- **Subject** (**TextEventManager**) виступає джерелом правди про стан тексту. Він не знає, хто саме його слухає, що відповідає принципу слабкого зв'язку.
- **Observers** (**StatusBarObserver**, **AutoSaveObserver**) підписуються на зміни суб'єкта. Це дозволяє додавати нові функції (наприклад, валідацію синтаксису) без зміни коду самого редактора.

Проблеми, які вирішує патерн:

- **Зменшення зв'язності (Decoupling):** Головна форма не повинна знати про всі можливі способи обробки тексту (статистика, логування). Вона лише передає текст менеджеру.
- **Динамічність:** Спостерігачів можна додавати або видаляти під час виконання програми (runtime).

- Принцип відкритості/закритості (ОСР): Систему легко розширювати новими класами-спостерігачами без модифікації існуючого коду.

Переваги використання:

- Можливість реактивного оновлення декількох частин інтерфейсу одночасно.
- Чіткий розподіл відповідальності між класами бізнес-логіки та UI.

Код:

```
1      using System;
2
3      namespace TextEditor.Observers
4      {
5          public interface IObserver
6          {
7              void Update(string textContent);
8          }
9      }
```

Рис. 3.1 – IObserver.cs

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace TextEditor.Observers
5  {
6      public class TextEventManager : ISubject
7      {
8          private readonly List<IObserver> _observers = new List<IObserver>();
9          private string _textContent;
10
11         public string TextContent
12         {
13             get { return _textContent; }
14             set
15             {
16                 if (_textContent != value)
17                 {
18                     _textContent = value;
19                 }
20             }
21         }
22
23         public void Attach(IObserver observer)
24         {
25             _observers.Add(observer);
26         }
27
28         public void Detach(IObserver observer)
29         {
30             _observers.Remove(observer);
31         }
32
33         public void Notify()
34         {
35             foreach (var observer in _observers)
36             {
37                 observer.Update(_textContent);
38             }
39         }
40     }
41 }

```

Рис. 3.2 – TextEventManager.cs

```

using System;
using System.Windows.Forms;

namespace :TextEditor.Observers
{
    public class StatusBarObserver : IObserver
    {
        private readonly ToolStripStatusLabel _statusLabel;

        public StatusBarObserver(ToolStripStatusLabel statusLabel)
        {
            _statusLabel = statusLabel;
        }

        public void Update(string textContent)
        {
            if (textContent == null) return;

            int charCount = textContent.Length;

            int wordCount = string.IsNullOrEmpty(textContent)
                ? 0
                : textContent.Split(new[] { ' ', '\n', '\r' }, StringSplitOptions.RemoveEmptyEntries).Length;

            _statusLabel.Text = $"Chars: {charCount} | Words: {wordCount} | Updated: {DateTime.Now:HH:mm:ss}";
        }
    }
}

```

Рис. 3.3 – StatusBarObserver.cs

```

1      using System;
2      using System.Diagnostics;
3
4      namespace TextEditorMK.Observers
5      {
6          public class AutoSaveObserver : IObserver
7          {
8              public void Update(string textContent)
9              {
10                 try
11                 {
12                     Debug.WriteLine($"[AutoSaveObserver] {DateTime.Now}: Зміни зафіксовано. Довжина тексту: {textContent.Length}");
13                 }
14                 catch (Exception ex)
15                 {
16                     Debug.WriteLine($"Error saving: {ex.Message}");
17                 }
18             }
19         }
20     }
21

```

Рис. 3.4 – AutoSaveObserver.cs


```

1      using System;
2
3      namespace TextEditor.Observers
4      {
5          public interface ISubject
6          {
7              void Attach(IObserver observer);
8              void Detach(IObserver observer);
9              void Notify();
10         }
11     }

```

Рис. 3.5 – ISubject.cs

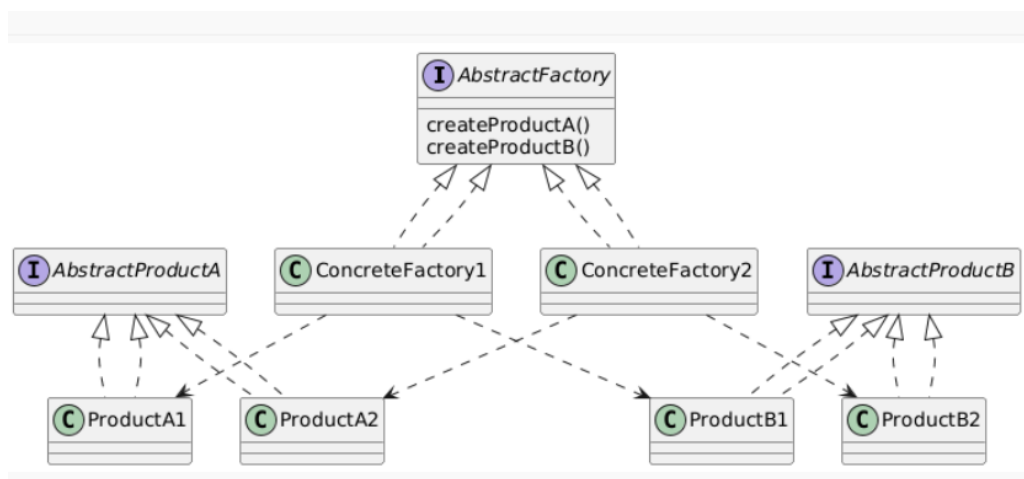
Висновок: У ході роботи було створено проєкт текстового редактора з використанням шаблону Observer. Реалізовано основні класи: IObserver та ISubject - інтерфейси для визначення спільної поведінки підписки та механізму сповіщення; TextEventManager - центральний клас-суб'єкт, що керує станом тексту та автоматично ініціює оновлення підписників при внесенні змін; StatusBarObserver та AutoSaveObserver - конкретні спостерігачі, що інкапсулюють логіку відображення статистики та фонового збереження даних. Застосування патерну Observer дозволило досягти слабкої зв'язності між компонентом зберігання даних та шаром відображення, забезпечило реактивність інтерфейсу в реальному часі, зробило систему гнучкою для додавання нових типів обробників подій та спростило підтримку коду.

Відповіді на контрольні питання:

1. Призначення шаблону «Абстрактна фабрика»

Шаблон Абстрактна фабрика використовується для створення сімейств пов'язаних об'єктів без прив'язки до конкретних класів, забезпечуючи узгодженість продуктів між собою.

2. Нарисуйте структуру шаблону «Абстрактна фабрика».



3. Які класи входять в шаблон «Абстрактна фабрика», та яка між ними взаємодія?

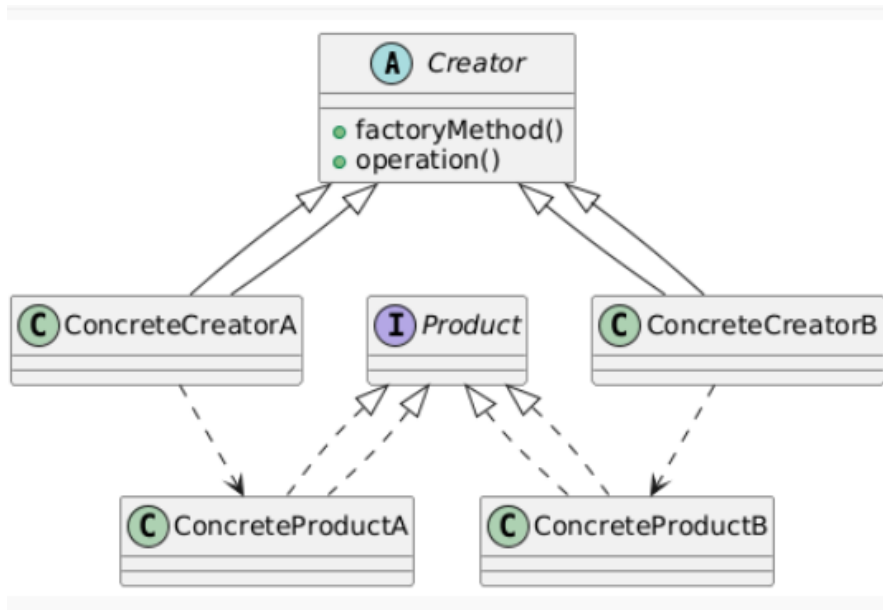
- **AbstractFactory** - інтерфейс фабрики
- **ConcreteFactory** - конкретні фабрики
- **AbstractProduct** - загальні інтерфейси продуктів
- **ConcreteProduct** - реалізації продуктів

Клієнт викликає методи абстрактної фабрики, яка повертає відповідні узгоджені об'єкти.

4. Яке призначення шаблону «Фабричний метод»?

Створює об'єкти через делегування створення підкласам, дозволяючи змінювати тип об'єктів без зміни коду клієнта.

5. Нарисуйте структуру шаблону «Фабричний метод».



6. Які класи входять в шаблон «Фабричний метод», та яка між ними взаємодія?

- **Creator** - містить фабричний метод
- **ConcreteCreator** - реалізує фабричний метод
- **Product** - базовий інтерфейс продукту
- **ConcreteProduct** - конкретні продукти

Creator викликає `factoryMethod()`, який визначає підклас.

7. Чим відрізняється шаблон «Абстрактна фабрика» від «Фабричний метод»?

Шаблони **Абстрактна фабрика** та **Фабричний метод** обидва належать до породжуючих шаблонів, але застосовуються в різних ситуаціях та відрізняються підходом до створення об'єктів.

Абстрактна фабрика використовується тоді, коли потрібно створювати групу взаємопов'язаних або взаємосумісних об'єктів. Головна ідея - об'єкти створюються фабрикою, яка повертає цілу "лінійку" продуктів, що логічно

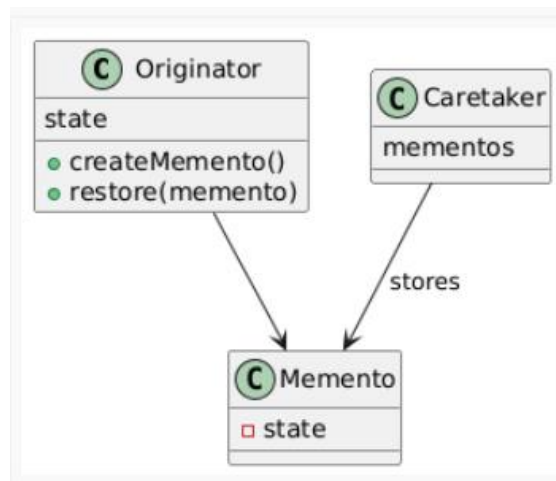
працюють разом. Клієнт не знає конкретних класів і працює лише з інтерфейсами. При зміні фабрики автоматично змінюються всі продукти, які вона створює.

Фабричний метод, на відміну від цього, зосереджений на створенні одного типу продукту, і сам процес створення делегується підкласам через перевизначення методу. Основна ідея -дати можливість змінювати тип створюваного об'єкта без зміни базового класу, використовуючи спадкування.

8. Яке призначення шаблону «Знімок»?

Зберігає стан об'єкта без порушення інкапсуляції, дозволяючи відкотитися назад.

9. Нарисуйте структуру шаблону «Знімок».



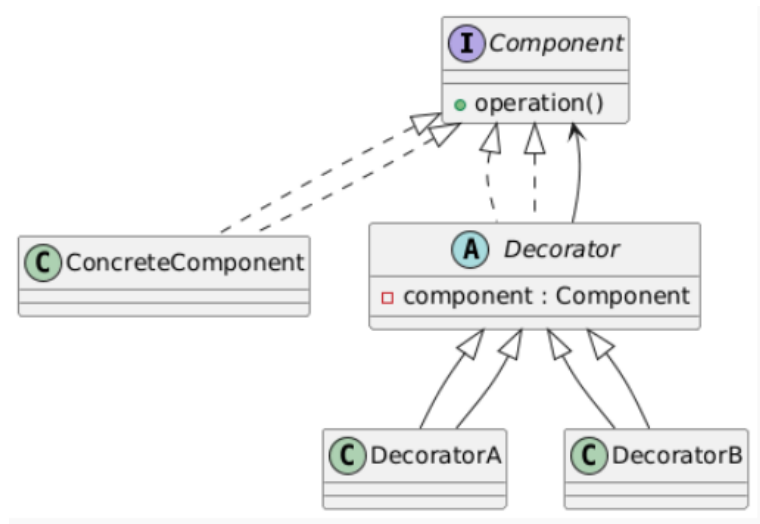
10. Які класи входять в шаблон «Знімок», та яка між ними взаємодія?

- Originator - створює та відновлює знімки
- Memento - зберігає стан
- Caretaker - керує історією станів

11. Яке призначення шаблону «Декоратор»?

Додає нову функціональність об'єктам без зміни їхнього коду та без наслідування.

12. Нарисуйте структуру шаблону «Декоратор».



13. Які класи входять в шаблон «Декоратор», та яка між ними взаємодія?

- **Component** - базовий інтерфейс
- **ConcreteComponent** - основний об'єкт
- **Decorator** - містить посилання на **Component**
- **ConcreteDecorator** - додають поведінку

14. Які є обмеження використання шаблону «декоратор»?

- Багато декораторів → складна структура
- Важко дебажити через ланцюг обгортки
- Не підходить, якщо треба змінювати поведінку всім об'єктам одразу